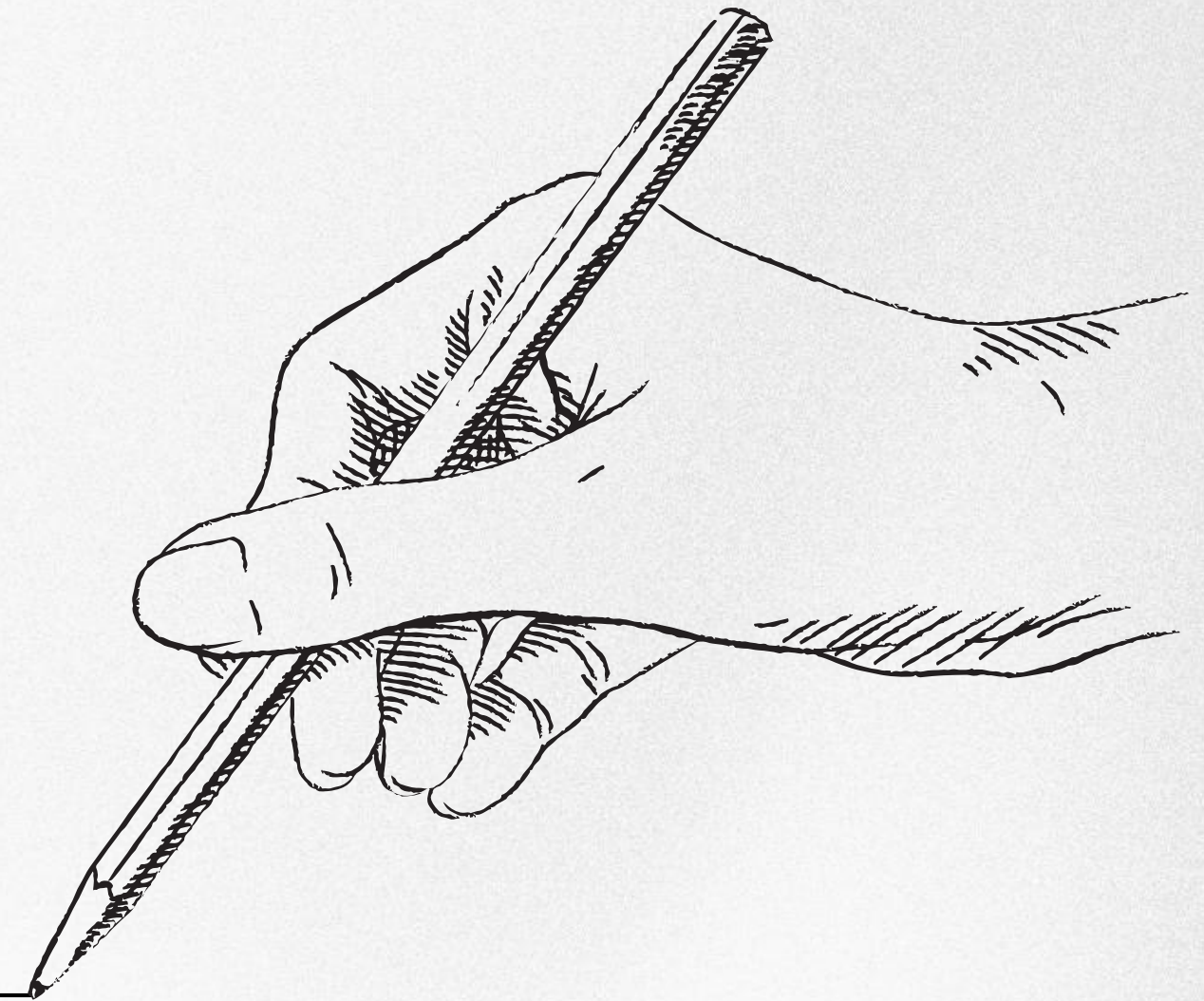


R 세미나

10.4

**Case study: byte
pair encoding**



BPE tokenizer

Byte pair encoding(BPE)

- 문장 or 단어 안에 있는 글자들을 적절한 단위로 나누는 subword tokenizer
- Token들의 빈도를 기반으로 높은 빈도의 토큰들을 merge하며 최종 token들을 만들어내는 방법

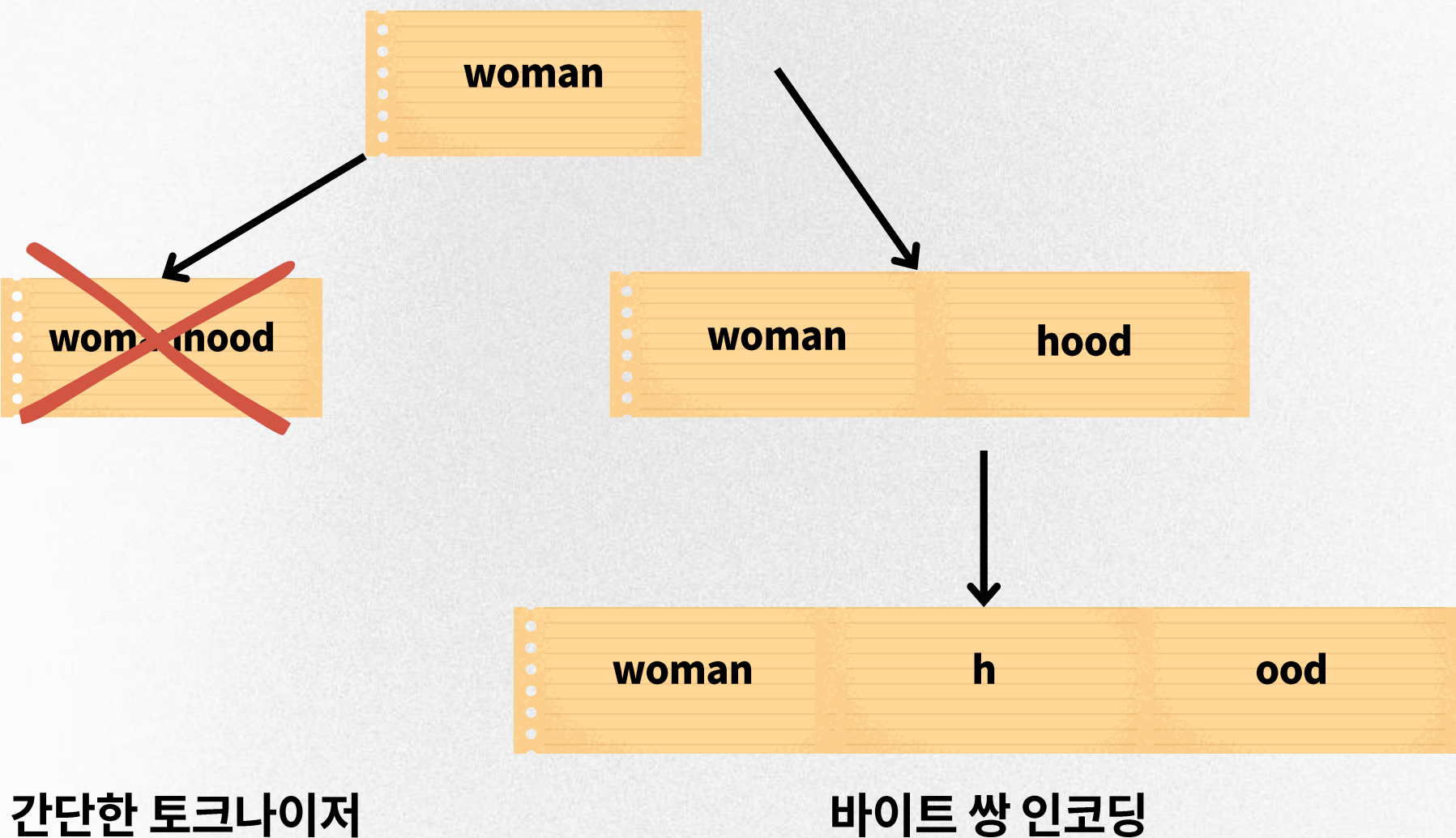
언어	단어	subword 구성
영어	concentrate	con(=together) + centr(=center) + ate(=make)
한국어	집중	集(모을 집)+ 中(가운데 중)

예시 : 실제로 영어나 한국어는 subword 단위로 의미를 분리할 수 있는 경우가 많음

책에서 소개하는 예시

바이트 쌍 인코딩은 문자 수준 정보와 단어 수준 정보 사이에 좋은 균형을 제공하며 알려지지 않은 단어도 인코딩할 수 있습니다.

바이트 쌍 인코딩은 모르는 단어로 인해 문제를 푸는 것이 까다로워지는 OOV(Out-Of-Vocabulary)문제를 해결하기 위해 사용됩니다.



예를 들어, 모델이 "woman"이라는 단어를 알고 있다고 가정합니다. 간단한 토큰나이저는 "womanhood"와 같은 단어를 알려지지 않는 버킷에 넣거나 완전히 무시하여 OOV 문제가 발생합니다.

바이트 쌍 인코딩은 "woman"과 "hood" (또는 모델이 "hood"를 충분히 일반적인 하위 단어로 찾았는지 여부에 따라 "woman", "h", "ood")와 같은 하위 단어를 선택할 수 있어야 합니다.

BPE tokenizer

알고리즘 예시1

1. 아래와 같은 문자열이 주어짐

aaabdaaabac

2. 위의 문자열 중 가장 자주 등장하고 있는 바이트의 쌍(byte pair)은 'aa'
이 'aa'라는 바이트의 쌍을 하나의 바이트인 'Z'로 치환

ZabdZabac
Z = aa

3. 위 문자열 중에서 가장 많이 등장하고 있는 바이트의 쌍은 'ab'
이 'ab'를 'Y'로 치환

ZYdZYac
Y = ab
Z = aa

4. 가장 많이 등장하는 바이트 쌍인 'ZY'를 'X'로 치환

XdXac
X = ZY
Y = ab
Z = aa

BPE tokenizer

알고리즘 예시2

```
# dictionary
# 훈련 데이터에 있는 단어와 등장 빈도수
low : 5, lower : 2, newest : 6, widest : 3
```

각 pre-tokenize로 나누어진 단어들의 빈도 계산

dictionary
low:5
lower:2
newest:6
widest:3

BPE 알고리즘은 기본적으로 문장이 pre-tokenize 되어 있다고 가정함.
즉, 하나의 문장이 모두 이어져 있는 것이 아닌 띄어쓰기 등으로 분리되어 있다고 가정.

→ 다음으로, 각 단어들을 하나의 글자 (character) 단위로 나눈다.

vocabulary
l, o, w, e, r, n, w, s, t, i, d

BPE tokenizer

알고리즘 예시2

빈도수가 7로 가장 높은 (l,o)의 쌍을 lo로 통합

'lo', w : 5
'lo' w e r : 2
n e w e s t : 6
w i d e s t : 3

vocabulary update
l,o,w,e,r,n,w,s,t,i,d,e,s,e,s,t, lo

딕셔너리를 참고하여, 빈도수가 9로 가장 높은 (e,s)의 쌍을 es로 통합

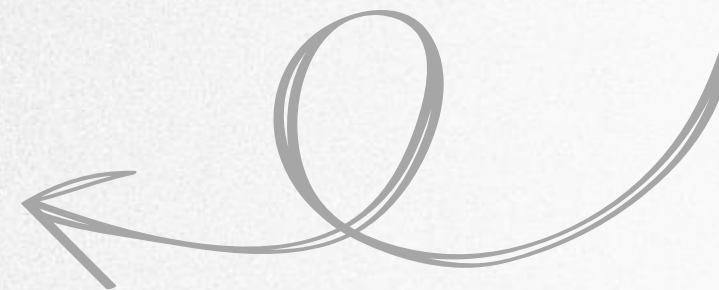
l o w : 5
l o w e r : 2
n e w 'e s' t : 6
w i d 'e s' t : 3

vocabulary update
l,o,w,e,r,n,w,s,t,i,d,e,s

빈도수가 9로 가장 높은 (es, t)의 쌍을 est로 통합

l o w : 5
l o w e r : 2
n e w 'e s t' : 6
w i d 'e s t' : 3

vocabulary update
l,o,w,e,r,n,w,s,t,i,d,e,s,e,s,t, est



BPE tokenizer

알고리즘 예시2

이와 같은 방식으로 총 10번 반복했을 때 얻은 딕셔너리와 단어 집합은 아래와 같음

low : 5
low e r : 2
newest : 6
widest : 3

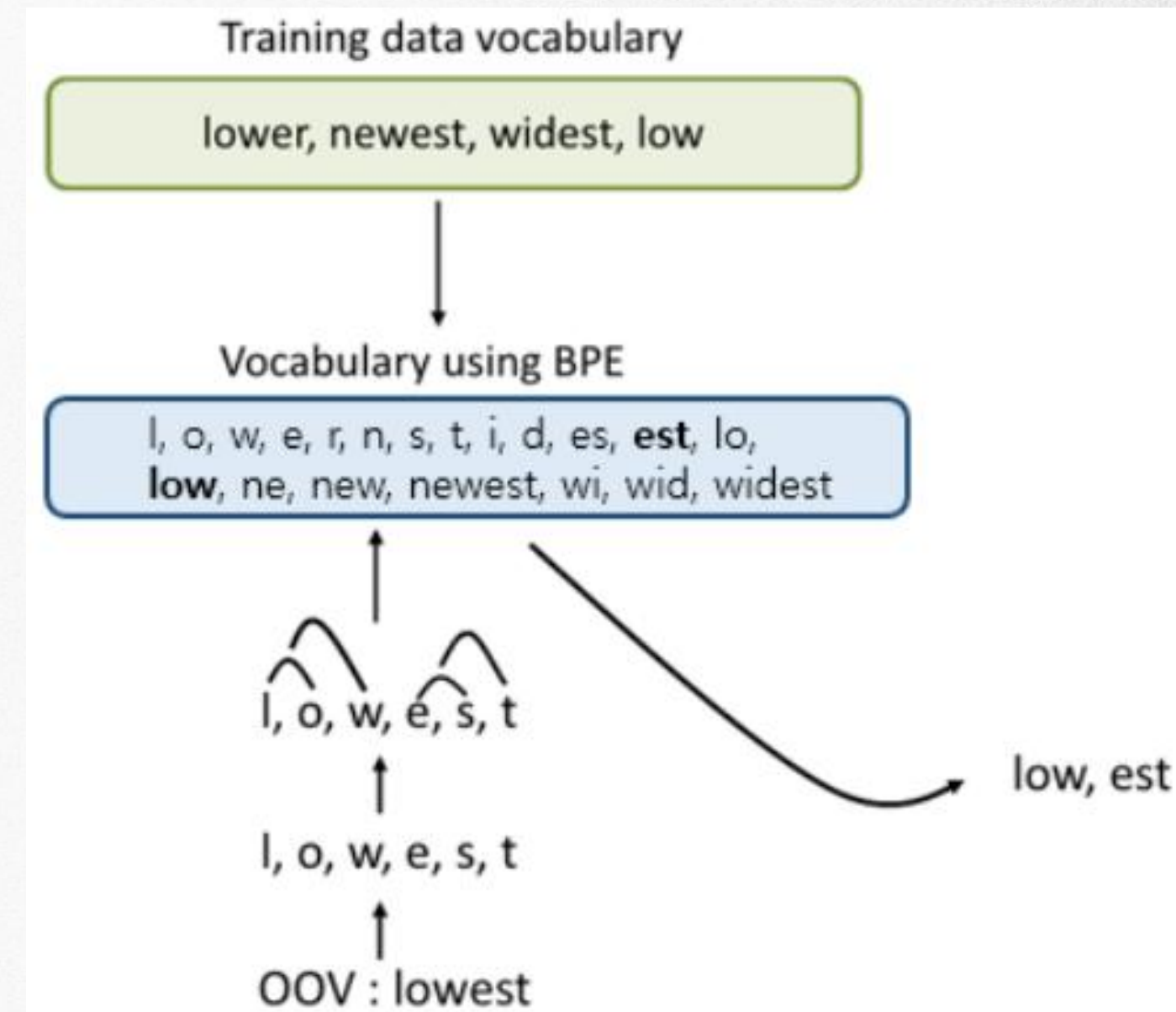
vocabulary update

l,o,w,e,r,n,w,s,t,i,d,es,est, lo, low, ne, new, newest, wi, wid, widest

이 경우 테스트 과정에서 'lowest'란 단어가 등장할 때
기존 : OOV에 해당되는 단어가 되었음

BPE 이후 : 'lowest'를 전부 글자 단위로 분할함.

- 'l, o, w, e, s, t'가 나옴.
- 기계는 위의 단어 집합을 참고하여 'low'와 'est'를 찾아냄.



BPE 동작 과정 예시

장단점 및 문제 상황을 위한 제안 내용

BPE 장.단점

- 장점 : 간단한 구조를 가지고 있으며, 빠른 속도로 작동, 추론 과정을 간소화하며 계산 효율성을 높일 수 있음, 모델의 정확도를 향상시킬 수 있음, 모델의 유연성을 확보할 수 있음
- 단점 : pre-tokenizing을 가정하고 있기 때문에 띄어쓰기가 없는 중국어와 일본어 등의 언어에 대해서는 적용이 힘들

제안내용

책에서 나오는 이 모델들은 전혀 언어 지식을 포함하지 않다.

- 훈련 세트에서 단지 문자 시퀀스의 형태론적 패턴만을 ‘학습’함
- 주어진 모델이 할 수 있는 것에 대한 기대치를 설정해야함

사용하는 패키지 : textrecipes

- 해당 패키지에는 바이트 쌍 인코딩을 수행하는 토큰화 엔진이 있음
- 어휘 크기와 적절한 시퀀스 길이를 결정해야함

function 함수를 사용하여 생성

코드설명

```
get_bpe_token_dist <- function(vocab_size, x) {  
  recipe(~text, data = tibble(text = x)) %>%  
  step_mutate(text = tolower(text)) %>%  
  step_tokenize(text,  
    engine = "tokenizers.bpe",  
    training_options = list(vocab_size = vocab_size)) %>%  
  prep() %>%  
  bake(new_data = NULL) %>%  
  transmute(n_tokens = lengths(textrecipes::get_tokens(text)),  
    vocab_size = vocab_size)  
}
```

1.vocab_size: BPE 토크나이저의 어휘 크기, x: 텍스트 데이터의 벡터

2.텍스트를 소문자로 변환하는 변환 단계(step_mutate)

3.BPE 토크나이저를 사용하여 텍스트를 토큰화, 여기서 vocab_size는
BPE 토크나이저의 어휘 크기를 지정하는 옵션

(engine, training_options)

4.준비된 레시피를 데이터에 적용(bake), new_data = NULL은 원래
데이터에 변환을 적용하겠다는 의미

5.텍스트에서 토큰을 추출하고, 각 텍스트의 토큰 개수를 계산하여
n_tokens로 저장 + 어휘 크기(vocab_size)를 함께 반환
(transmute)

map 함수를 사용하여 사전크기 다루기

코드설명

```
bpe_token_dist <- map_dfr(  
  c(2500, 5000, 10000, 20000),  
  get_bpe_token_dist,  
  kickstarter_train$blurb  
)  
bpe_token_dist
```

1.map_dfr 함수를 사용하여 리스트의 각 요소에 함수를 적용, 결과를 데이터 프레임으로 결합

2.BPE 토크나이저의 어휘 크기로 사용할 값들의 벡터

: 각 토큰 크기가 텍스트 표현이 미치는 영향을 비교하기 위해서

(2500,5000,10000,20000)

Ex. n_token이 15라면 해당 캠페인 블럽은 BPE 토큰화된 후 15개의 서브워드로 분해

vocab_size가 2500개라면, 2500개의 서브워드를 사용하는 어휘 크기에서 BPE 토큰화가 수

행

```
# A tibble: 808,368 x 2  
  n_tokens vocab_size  
  <int>      <dbl>  
1      15      2500  
2      26      2500  
3      23      2500  
4      34      2500  
5      36      2500  
6      35      2500  
7      37      2500  
8      34      2500  
9      48      2500  
10     17      2500
```


그림으로 비교분석

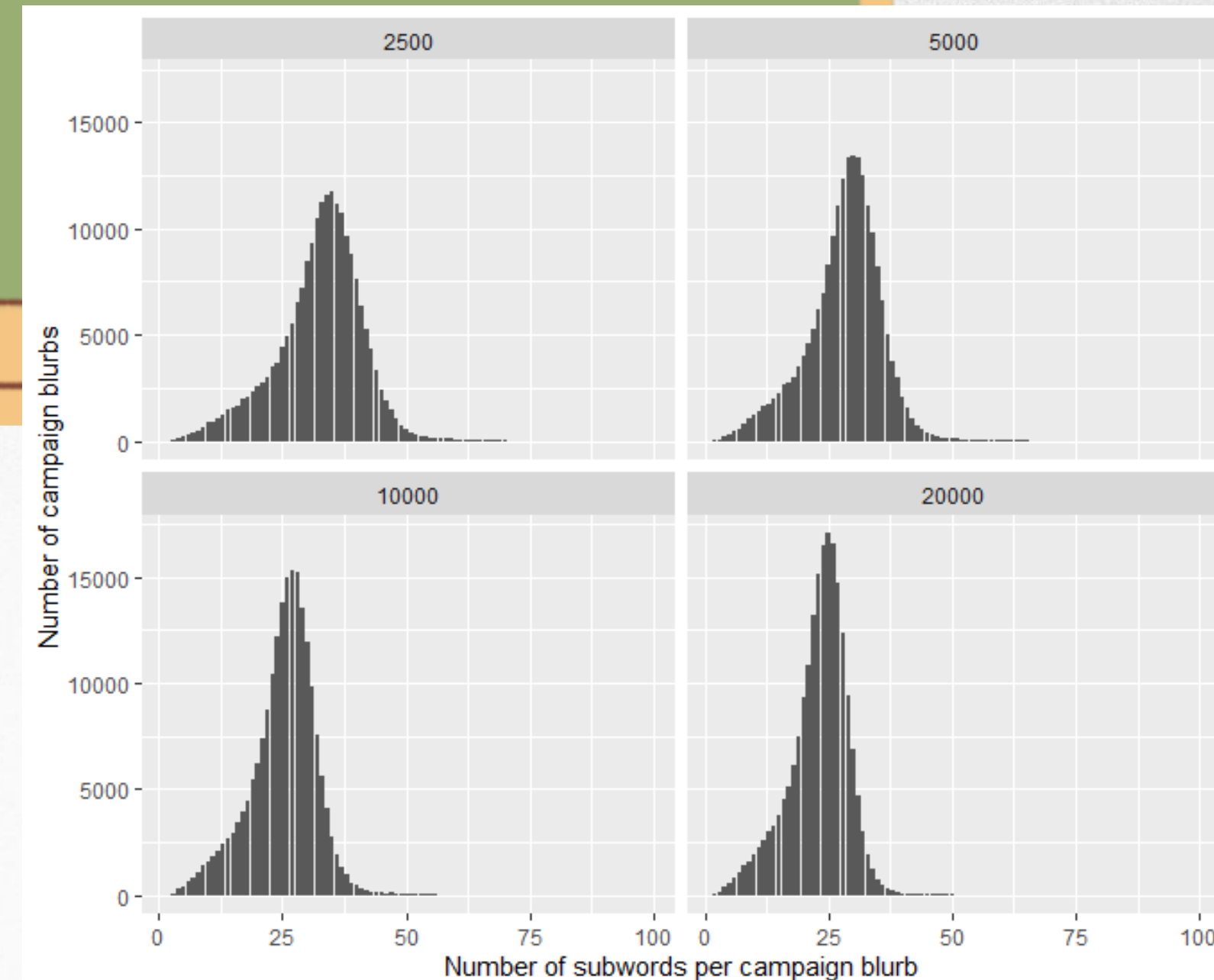
코드설명

```
bpe_token_dist %>%  
  ggplot(aes(n_tokens)) +  
  geom_bar() +  
  facet_wrap(~vocab_size) +  
  labs(x = "Number of subwords per campaign blurb",  
       y = "Number of campaign blurbs")
```

bpe_token_dist 데이터 프레임을 사용하여 각 어휘 크기(vocab_size)에 대해 텍스트의 토큰 개수(n_tokens)를 막대 그래프로 시각화

어휘 크기 2500 : 균일한 서브워드 분포를 제공, 데이터의 변동성을 잘 반영하여 일반화 성능을 높일 수 있음

어휘 크기 20000 : 대부분의 시퀀스가 비슷한 길이로 나타나 모델의 효율성 높임, 긴 시퀀스에 대한 대응이 어려움



전처리 단계 정의 및 준비

코드설명

```
bpe_rec <- recipe(~blurb, data = kickstarter_train) %>%  
  step_mutate(blurb = tolower(blurb)) %>%  
  step_tokenize(blurb,  
    engine = "tokenizers.bpe",  
    training_options = list(vocab_size = max_subwords)) %>%  
  step_sequence_onehot(blurb, sequence_length = bpe_max_length)  
  
bpe_prep <- prep(bpe_rec)
```

- 1.recipe함수로 텍스트 전처리 과정 정의
- 2.텍스트를 소문자로 변환하는 단계를 거침
- 3.step_tokenize 함수는 BPE 토크나이저를 사용하여 토큰화
- 4.training_options 인자로 vocab_size를 max_subwords로 설정
- 5.step_sequence_onehot 원-핫 인코딩을 진행
- 6.prep함수를 통해 레시피를 준비하여 데이터에 적용시킴

— Recipe —

— Inputs

Number of variables by role
predictor: 1

— Operations

- Variable mutation for: tolower(blurb)
- Tokenization for: blurb
- Sequence 1 hot encoding for: blurb

CNN 모델층 생성

코드설명

```
cnn_bpe <- keras_model_sequential() %>%  
  layer_embedding(input_dim = max_subwords + 1, output_dim = 16,  
    input_length = bpe_max_length) %>%  
  layer_conv_1d(filter = 32, kernel_size = 7, activation = "relu") %>%  
  layer_global_max_pooling_1d() %>%  
  layer_dense(units = 64, activation = "relu") %>%  
  layer_dense(units = 1, activation = "sigmoid")
```

Layer (type)	Output shape	Param #
embedding (Embedding)	(None, 40, 16)	160016
conv1d (Conv1D)	(None, 34, 32)	3616
global_max_pooling1d (GlobalMaxPooling1D)	(None, 32)	0
dense_1 (Dense)	(None, 64)	2112
dense (Dense)	(None, 1)	65
Total params: 165809 (647.69 KB)		
Trainable params: 165809 (647.69 KB)		
Non-trainable params: 0 (0.00 Byte)		

1. Embedding layer

paramter 계산식 : vocab_size * embedding_dim

vocab_size : 10,000

embedding_dim : 16

parameter = (10,000 * 16) + 16 = 160016

- input_dim을 max_subwords의 개수로 고정

2.Conv layer

paramter 계산식 : filter_size * input_dim * num_filters

filter_size : 7

input dim : 16

num_filter : 32

parameter : (7 * 16 * 32) + 32 = 3616

CNN 모델층 생성

코드설명

```
cnn_bpe <- keras_model_sequential() %>%  
  layer_embedding(input_dim = max_subwords + 1, output_dim = 16,  
    input_length = bpe_max_length) %>%  
  layer_conv_1d(filter = 32, kernel_size = 7, activation = "relu") %>%  
  layer_global_max_pooling_1d() %>%  
  layer_dense(units = 64, activation = "relu") %>%  
  layer_dense(units = 1, activation = "sigmoid")
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 40, 16)	160016
conv1d (Conv1D)	(None, 34, 32)	3616
global_max_pooling1d (GlobalMaxPooling1D)	(None, 32)	0
dense_1 (Dense)	(None, 64)	2112
dense (Dense)	(None, 1)	65

=====
Total params: 165809 (647.69 KB)
Trainable params: 165809 (647.69 KB)
Non-trainable params: 0 (0.00 Byte)

3.Max pooling층

output shape = 32
parameter = X

4.Dense층

input dim = 32
output dim = 64
parameter = $(32+1) * 64 = 2112$

5.Dense layer

input dim : 64
output dim : 1
parameter : $(64 + 1) * 1 = 65$

CNN 모델 훈련

코드설명

```
optimizer = "adam", loss = "binary_crossentropy", metrics = "accuracy"  
  
bpe_history <- cnn_bpe %>% fit(  
  bpe_analysis,  
  state_analysis,  
  epochs = 10,  
  validation_data = list(bpe_assess, state_assess),  
  batch_size = 512  
)
```


CNN 모델 훈련 결과

코드설명

	DNN	LSTM	CNN	BPE.CNN
Loss	0.03779	0.3891	0.03192	0.03477
Accuracy	0.9913	0.8148	0.9936	0.9937
Val_loss	1.023	0.6088	0.9301	0.9647
Val_accuracy	0.8075	0.7335	0.813	0.8161

Predict된 결과 확인

코드설명

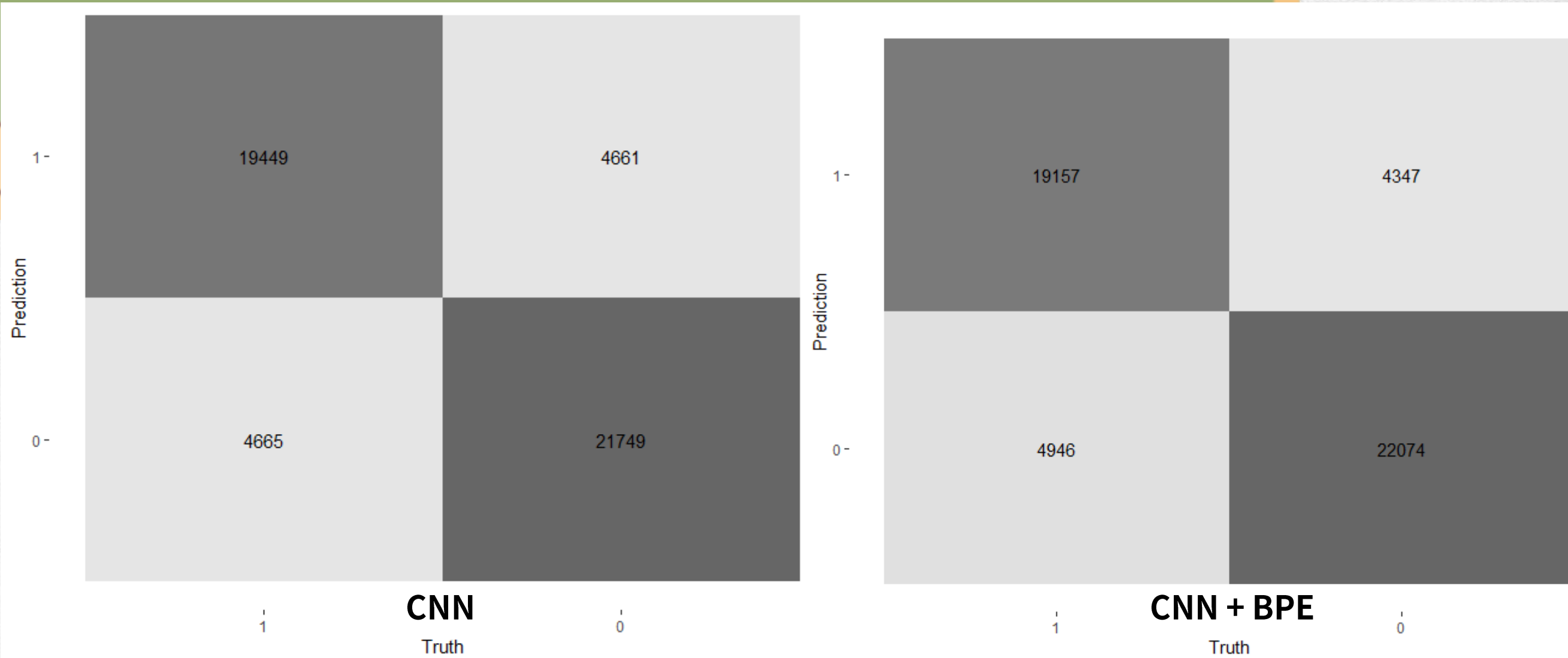
```
val_res_bpe <- keras_predict(cnn_bpe, bpe_assess, state_assess)

val_res_bpe %>%
  conf_mat(state, .pred_class) %>%
  autoplot(type = "heatmap")
```

혼합 행렬표로 확인

0,1 확인

	.pred_1	.pred_class	state
	<dbl>	<fct>	<fct>
1	1.00	1	1
2	1.00	1	1
3	0.000171	0	0
4	0.0126	0	0
5	0.00000316	0	0
6	1.00	1	1
7	0.0000288	0	0
8	0.0310	0	0
9	0.921	1	0
10	0.949	1	1



Predict된 결과 확인

코드설명

```
bpe_rec %>%  
  prep() %>%  
  tidy(3) %>%  
  filter(str_detect(token, "^h")) %>%  
  pull(token)
```

결과 의미 및 해석 : 출력된 결과는 문자 “h”로 시작하는 모든 토큰을 나타냄.

(“h”로 시작하는 다양한 서브워드들이 어떻게 생성되었는지, 원본 텍스트에서 자주 등장하는 패턴을 파악, 특정 문자로 시작하는 토큰들을 검토하여 토큰화의 정확성을 평가)

```
blurb8561 blurb8562 blurb8563 blurb8564 blurb8565 blurb8566 blurb8567 blurb8568 blurb8569 blurb8570 blurb8571 blurb8572 blurb8573 blurb8574  
"h"      "ha"      "hab"      "ham"      "hand"      "he"      "head"      "heart"      "heast"      "hed"      "heim"      "hel"      "help"      "hem"  
blurb8575 blurb8576 blurb8577 blurb8578 blurb8579 blurb8580 blurb8581 blurb8582 blurb8583 blurb8584 blurb8585 blurb8586 blurb8587 blurb8588  
"hen"      "her"      "here"      "hern"      "hero"      "hes"      "hes,"      "hes."      "hest"      "het"      "hetic"      "hett"      "hib"      "hic"  
blurb8589 blurb8590 blurb8591 blurb8592 blurb8593 blurb8594 blurb8595 blurb8596 blurb8597 blurb8598 blurb8599 blurb8600 blurb8601 blurb8602  
"hing"      "hing."      "hip"      "hist"      "hn"      "hol"      "hold"      "hood"      "hop"      "hor"      "hous"      "house"      "how"      "hr"  
blurb8603  
"hu"
```


Predict된 결과 확인

코드설명

```
bpe_rec %>%  
  prep() %>%  
  tidy(3) %>%  
  arrange(desc(nchar(token))) %>%  
  slice_head(n = 25) %>%  
  pull(token)
```

결과 의미 및 해석 : BPE 토큰화 과정에서 생성된 가장 긴 토큰 25개를 나타냄.

(아래에 나온 복합어는 텍스트에서 자주 사용되는 표현일 가능성이 높음, 어휘 다양성을 나타냄, 해당 텍스트는 음악과 영화 등 특정 주제와 관련된 용어들을 많이 포함하고 있다는 것을 알 수 있음)

blurb6317	blurb6321	blurb5386	blurb2620	blurb3842	blurb5387
"_singer-songwriter"	"_singer/songwriter"	"_post-apocalyptic"	"_environmentally"	"_interchangeable"	"_post-production"
blurb6320	blurb2604	blurb2870	blurb3340	blurb3684	blurb5515
"_singer/songwrit"	"_entertainment."	"_feature-length"	"_groundbreaking"	"_illustrations."	"_professionally"
blurb5792	blurb6156	blurb6745	blurb7068	blurb7183	blurb807
"_relationships."	"_self-published"	"_sustainability"	"_transformation"	"_unconventional"	"_architectural"
blurb937	blurb953	blurb1661	blurb1662	blurb1663	blurb1715
"_automatically"	"_award-winning"	"_collaborating"	"_collaboration"	"_collaborative"	"_coming-of-age"
blurb1730					
"_communication"					

요약

전처리된 텍스트 데이터에서 BPE 토큰화를 통해 생성된 토큰을 추출하여 텍스트의 복잡성과 어휘 다양성을 분석하는데 유용함.

이를 통해 원본 텍스트의 주요 주제와 특성을 이해하는데 도움을 줌.

책의 저자는 'CNN으로 학습하는 과정에서 이러한 방법도 있다'고 설명